MITRC

MODERN INSTITUTE OF TECHNOLOGY & RESEARCH CENTRE

NAME OF FACULTY:- Mr. Nitin Gandhi

SUBJECT/ SUB. CODE : programming with C Language (BCA-103)

SEMESTER:- I

SESSION:-2025-2026(Odd Sem)

BRANCH:-BCA

BATCH:-A

DATE OF SUBMITTION:-10-12-2025

Topic Covered:- Important Questions With Answer

Section A(40 Questions)

Section B(15 Questions)

Section C(15 Questions)

FACULTY

HOD

DEAN ACADE

Part A

Q1. What are the basic datatypes supported in C programming language?

Q2. What do you mean by the scope of a variable? What is the scope of variables in C?

Q3. What are static variables and functions?

Q4. Differentiate between calloc() and malloc().

Q5. What are the valid places where a programmer can apply the break control statement?

Q6. How can we store a negative integer?

Q7. Differentiate between actual parameters and formal parameters.

Q8. Can a C program be compiled or executed in the absence of a main()?

Q9. What do you mean by a nested structure?

Q10. What is a C token?

Q11. What is a preprocessor?

Q12. Why is C called the mother of all languages?

Q13. Mention the features of C programming language.

Q14. What is the purpose of printf() and scanf() in a C program?

Q15. What is an array?

Q16. What is the \0 character?

Q17. What is the main difference between a compiler and an interpreter?

Q18. Can I use int datatype to store the value 32768?

Q19. How is a function declared in C language?

Q20. What is dynamic memory allocation?

Q21. Where can we not use the & (address) operator in C?

Q22. Write a simple example of a structure in C language.

Q23. Differentiate between call by value and call by reference.

Q24. Differentiate between getch() and getche().

Q25. Explain toupper() with an example.

Q26. Explain local static variables and their use.

Q27. What is the difference between declaring a header file with < > and " "?

Q28. When should we use the register storage specifier?

Q29. Which statement is more efficient: x = x + 1 or x++?

Q30. Can we declare the same variable name for variables with different scopes?

Q31. Which operator is used to access union data members if a union variable is declared as a pointer?

Q32. Mention file operations in C language.

Q33. What are the different storage class specifiers in C?

Q34. What is typecasting?

Q35. Write a C program to print "hello world" without using a semicolon.

Q36. Write a program to swap two numbers without using a third variable.

Q37. How can you print a string with the symbol % in it?

Q38. Write a code to print a number pattern (like 1, 12, 123...).

Q39. What is a pointer in C and why is it used?

Q40. If a global and local variable have the same name, can we access the global variable inside the block?

Part B

Q1. What is Bubble Sort? Explain with a program.
Q2. Develop a C program to print the Fibonacci series.
Q3. Develop a C program to print a given pyramid pattern.
Q4. Write a program to add two numbers and explain the program using a flowchart.
Q5. What is a pointer? Discuss the advantages of pointers.
Q6. Explain pointer declaration with examples.
Q7. Write a C program to print the address of variables using pointers.
Q8. What are the benefits of using pointers in C?
Q9. Develop a program to find the largest number in an array.
Q10. Write a program to reverse a number in C.
Q11. Write a program to find the factorial of a number using recursion.
Q12. Write a program to check whether a number is prime or not.
Q13. Write a program to print multiplication tables.
Q14. Write a C program to count vowels and consonants in a string.
Q15. Write a program to read and write data into a file using file handling.

Part C

Q.1 What is Recursion? Write a program to enter a number and find its factorial using recursion. Also write a program for Fibonacci using recursion.

Q.2 Explain different categories of function with an example. And explain getch(),getche(),puts(),gets(),strcpy() and strcat().

Q.3 What is nested structure? There is a structure called employee that hold information like employee id, name and date of joining. Write a program to create an array of structure and enter some date into it. Then ask the user to enter name. Display the record of those employee whose name is match to the enter name.

Q.4 (i)Write a program to write and read record of employee using fread() and fwrite() function.

(ii) Write a program to write and read record of student using fscanf() and fprintf() function.

(iii) write a program to open a pre existing file and add information at the end of the file. Display the contents of the file before and after appending.

Q.5 Short notes on:

(i) fseek() (ii) ftell() (iii)rewind() (iv) feof()

Q.6 (i)Write a program to enter employee record using pointer to structure.

(ii) Write a program to enter student record using structure and function.

Q7 Write a program to copy the content of one file to another file.

Q 8. Describe Pointer to function and pointer and function with example .

Q.9Describe the following?

(a)Enumerated data type

(b) Operators

Q.10 Short notes on:

(a) Command line argument (b) void pointer (c) typedef, sizeof

(d)while and do while (e) dynamic memory allocation (f) constants in C

Q.11

(a) Write a program to enter a string and count length of string and reverse string.

(b) Write a program to enter a year and check it is leap year or not.

Q.12 Write a program to Accessing an array element using pointer.

Q.13

(a) Describe array & pointer?

(b) What is the difference between array, structure, union and pointer.

Q.14 Write a program to passing array to function.

Q.15 Explain call by value and call by reference with an example.

PART-A

Q1. What are the basic Datatypes supported in C Programming Language?

Ans: The Datatypes in C Language are broadly classified into 4 categories. They are as follows:

- Basic Datatypes
- Derived Datatypes
- Enumerated Datatypes
- Void Datatypes

The Basic Datatypes supported in C Language are as follows:

Datatype Name	Datatype Size	Datatype Range
Short	1 byte	-128 to 127
unsigned short	1 byte	0 to 255
Char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
Int	2 bytes	-32,768 to 32,767
unsigned int	2 bytes	0 to 65,535
Long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295
Float	4 bytes	3.4E-38 to 3.4E+38
Double	8 bytes	1.7E-308 to 1.7E+308
long double	10 bytes	3.4E-4932 to 1.1E+4932

Q2. What do you mean by the Scope of the variable? What is the scope of the variables in C?

Ans: Scope of the variable can be defined as the part of the code area where the variables declared in the program can be accessed directly. In C, all identifiers are lexically (or statically) scoped.

Q3. What are static variables and functions?

Ans: The variables and functions that are declared using the keyword Static are considered as Static Variable and Static Functions. The variables declared using Static keyword will have their scope restricted to the function in which they are declared.

Q4. Differentiate between calloc() and malloc()

Ans: calloc() and malloc() are memory dynamic memory allocating functions. The only difference between them is that calloc() will load all the assigned memory locations with value 0 but malloc() will not.

Q5. What are the valid places where the programmer can apply Break Control Statement?

Ans: Break Control statement is valid to be used inside a loop and Switch control statements.

Q6. How can we store a negative integer?

Ans: To store a negative integer, we need to follow the following steps. Calculate the two's complement of the same positive integer.

Eg: 1011 (-5)

Step-1 − One's complement of 5: 1010

Step-2 − Add 1 to above, giving 1011, which is -5

Q7. Differentiate between Actual Parameters and Formal Parameters.

Ans: The Parameters which are sent from main function to the subdivided function are called as Actual Parameters and the parameters which are declared a the Subdivided function end are called as Formal Parameters.

Q8. Can a C program be compiled or executed in the absence of a main()?

Ans: The program will be compiled but will not be executed. To execute any C program, main() is required.

Q9. What do you mean by a Nested Structure?

Ans: When a data member of one structure is referred by the data member of another function, then the structure is called a Nested Structure.

Q10. What is a C Token?

Ans: *Keywords, Constants, Special Symbols, Strings, Operators, Identifiers* used in C program are referred to as C Tokens.

Q11. What is Preprocessor?

Ans: A Preprocessor Directive is considered as a built-in predefined function or macro that acts as a directive to the compiler and it gets executed before the actual C Program is executed.

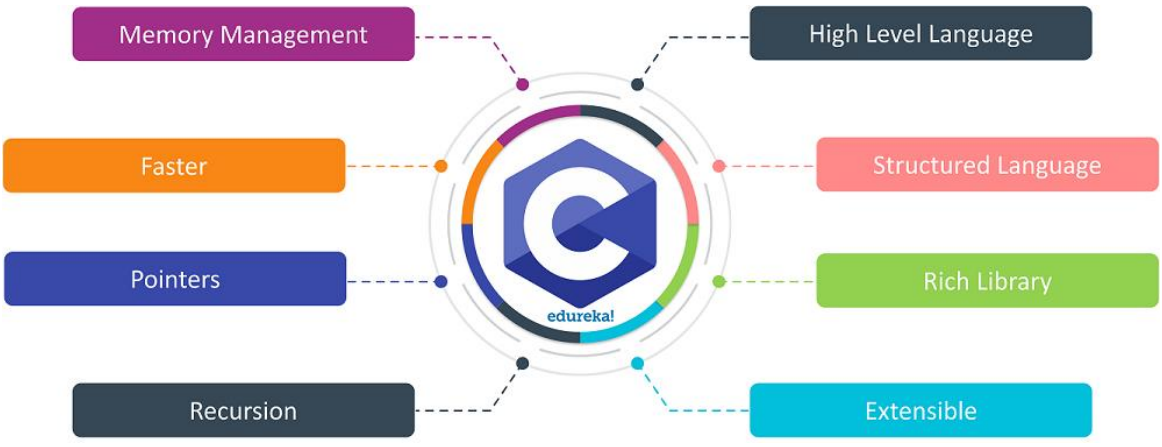
In case you are facing any challenges with these C Programming Interview Questions, please write your problems in the comment section below.

Q12. Why is C called the Mother of all Languages?

Ans: C introduced many core concepts and data structures like arrays, lists, functions, strings, etc. Many languages designed after C are designed on the basis of C Language. Hence, it is considered as the mother of all languages.

Q13. Mention the features of C Programming Language.

Ans:



Q14. What is the purpose of printf() and scanf() in C Program?

Ans: printf() is used to print the values on the screen. To print certain values, and on the other hand, scanf() is used to scan the values. We need an appropriate datatype format specifier for both printing and scanning purposes. For example,

- %d: It is a datatype format specifier used to print and scan an integer value.
- %s: It is a datatype format specifier used to print and scan a string.
- %c: It is a datatype format specifier used to display and scan a character value.
- %f: It is a datatype format specifier used to display and scan a float value.

Q15. What is an array?

Ans. The array is a simple data structure that stores multiple elements of the same datatype in a reserved and sequential manner. There are three types of arrays, namely,

- One Dimensional Array
- Two Dimensional Array
- Multi-Dimensional Array

Q16. What is \0 character?

Ans: The Symbol mentioned is called a Null Character. It is considered as the terminating character used in strings to notify the end of the string to the compiler.

Q17. What is the main difference between the Compiler and the Interpreter?

Ans: Compiler is used in C Language and it translates the complete code into the Machine Code in one shot. On the other hand, Interpreter is used in Java Programming Langauge and other high-end programming languages. It is designed to compile code in line by line fashion.

Q18. Can I use int datatype to store 32768 value?

Ans: No, Integer datatype will support the range between -32768 and 32767. Any value exceeding that will not be stored. We can either use float or long int.

Q19. How is a Function declared in C Language?

Ans: A function in C language is declared as follows,

```
1  return_type function_name(formal parameter list)
2  {
```

```
3       Function_Body;
4   }
```

Q20. What is Dynamic Memory allocation? Mention the syntax.

Ans: Dynamic Memory Allocation is the process of allocating memory to the program and its variables in runtime. Dynamic Memory Allocation process involves three functions for allocating memory and one function to free the used memory.

malloc() – Allocates memory

Syntax:

```
1 ptr = (cast-type*) malloc(byte-size);
```

calloc() – Allocates memory

Syntax:

```
1 ptr = (cast-type*)calloc(n, element-size);
```

realloc() – Allocates memory

Syntax:

```
1 ptr = realloc(ptr, newsize);
```

free() – Deallocates the used memory

Syntax:

```
1 free(ptr);
```

Q21.Where can we not use &(address operator in C)?

Ans: We cannot use & on constants and on a variable which is declared using the register storage class.

Q22. Write a simple example of a structure in C Language

Ans: Structure is defined as a user-defined data type that is designed to store multiple data members of the different data types as a single unit. A structure will consume the memory equal to the summation of all the data members.

```
1  struct employee
2  {
3      char name[10];
4      int age;
5  }e1;
6  int main()
7  {
8      printf("Enter the name");
9      scanf("%s",e1.name);
10     printf("n");
11     printf("Enter the age");
12     scanf("%d",&amp;amp;amp;e1.age);
13     printf("n");
14     printf("Name and age of the employee: %s,%d",e1.name,e1.age);
15     return 0;
16}
```

Q23. Differentiate between call by value and call by reference.

Ans:

Factor	Call by Value	Call by Reference
Safety	Actual arguments cannot be changed and remain safe	Operations are performed on actual arguments, hence not safe
Memory Location	Separate memory locations are created for actual and formal arguments	Actual and Formal arguments share the same memory space.
Arguments	Copy of actual arguments are sent	Actual arguments are passed

//Example of Call by Value method

```
1  #include<stdio.h>
2  void change(int,int);
3  int main()
4  {
5      int a=25,b=50;
```

```
6     change(a,b);
7     printf("The value assigned to a is: %d",a);
8     printf("n");
9     printf("The value assigned to of b is: %d",b);
10    return 0;
11}
12void change(int x,int y)
13{
14    x=100;
15    y=200;
16}
```

//Output

The value assigned to of a is: 25
The value assigned to of b is: 50

//Example of Call by Reference method

```
1  #include<stdio.h>
2  void change(int*,int*);
3  int main()
4  {
5     int a=25,b=50;
6     change(&amp;amp;a,&amp;amp;b);
7     printf("The value assigned to a is: %d",a);
8     printf("n");
9     printf("The value assigned to b is: %d",b);
10    return 0;
11}
12void change(int *x,int *y)
13{
14    *x=100;
15    *y=200;
16}
```

//Output

The value assigned to a is: 100
The value assigned to b is: 200

In case you are facing any challenges with these C Programming Interview Questions, please write your problems in the comment section below.

Q24. Differentiate between getch() and getche().

Ans: Both the functions are designed to read characters from the keyboard and the only difference is that

getch(): reads characters from the keyboard but it does not use any buffers. Hence, data is not displayed on the screen.

getche(): reads characters from the keyboard and it uses a buffer. Hence, data is displayed on the screen.

//Example

```
1  #include<stdio.h>;
2  #include<conio.h>
3  int main()
4  {
5      char ch;
6      printf("Please enter a character ");
7      ch=getch();
8      printf("nYour entered character is %c",ch);
9      printf("nPlease enter another character ");
10     ch=getche();
11     printf("nYour new character is %c",ch);
12     return 0;
13}
```

//Output

```
Please enter a character
Your entered character is x
Please enter another character z
Your new character is z
```

Q25. Explain toupper() with an example.

Ans. toupper() is a function designed to convert lowercase words/characters into upper case.

//Example

```
1#include<stdio.h>
2#include<conio.h>
3int main()
4{
5   char c;
6   c=a;
7   printf("%c after conversions  %c", c, toupper(c));
8   c=B;
9   printf("%c after conversions  %c", c, toupper(c));
```

//Output:

a after conversions A
B after conversions B

Q26. Explain Local Static Variables and what is their use?

Ans: A local static variable is a variable whose life doesn't end with a function call where it is declared. It extends for the lifetime of the complete program. All calls to the function share the same copy of local static variables.

```
1  #include<stdio.h>
2
3  void fun()
4  {
5     static int x;
6     printf("%d ", x);
7     x = x + 1;
8  }
9  int main()
10 {
11    fun();
12    fun();
13    return 0;
   }
```

//Output

0 1

Q27. What is the difference between declaring a header file with < > and ” “?

Ans: If the Header File is declared using < > then the compiler searches for the header file within the Built-in Path. If the Header File is declared using ” ” then the compiler will search for the Header File in the current working directory and if not found then it searches for the file in other locations.

Q28. When should we use the register storage specifier?

Ans: We use Register Storage Specifier if a certain variable is used very frequently. This helps the compiler to locate the variable as the variable will be declared in one of the CPU registers.

Q29. Which statement is efficient and why? x=x+1; or x++; ?

Ans: x++; is the most efficient statement as it just a single instruction to the compiler while the other is not.

Q30. Can I declare the same variable name to the variables which have different scopes?

Ans: Yes, Same variable name can be declared to the variables with different variable scopes as the following example.

```
1int var;
2void function()
3{
4   int variable;
5}
6int main()
7{
8   int variable;
9}
```

Q31. Which variable can be used to access Union data members if the Union variable is declared as a pointer variable?

Ans: Arrow Operator(->) can be used to access the data members of a Union if the Union Variable is declared as a pointer variable.

Q32. Mention File operations in C Language.

Ans: Basic File Handling Techniques in C, provide the basic functionalities that user can perform against files in the system.

Function	Operation
fopen()	To Open a File
fclose()	To Close a File
fgets()	To Read a File
fprint()	To Write into a File

In case you are facing any challenges with these C Programming Interview Questions, please write your problems in the comment section below.

Q33. What are the different storage class specifiers in C?

Ans: The different storage specifiers available in C Language are as follows:

- auto
- register
- static
- extern

Q34. What is typecasting?

Ans: Typecasting is a process of converting one data type into another is known as typecasting. If we want to store the floating type value to an int type, then we will convert the data type into another data type explicitly.

Syntax:

1(type_name) expression;

Q35. Write a C program to print hello world without using a semicolon (;).

Ans:

```
1       #include<stdio.h>
2
3  void main()
```

```
4{
5     if(printf("hello world")){}
  }
```

//Output:

hello world

Q36. Write a program to swap two numbers without using the third variable.

Ans:

```
1
2  #include<stdio.h>
3   main()
4   {
5       int a=10, b=20;
6       clrscr();
7       printf("Before swapping a=%d b=%d",a,b);
8       a=a+b;
9       b=a-b;
10      a=a-b;
11      printf("nAfter swapping a=%d b=%d",a,b);
12      getch();
13  }
```

//Output

Before swapping a=10 b=20
After swapping a=20 b=10

Q37. How can you print a string with the symbol % in it?

Ans: There is no escape sequence provided for the symbol % in C. So, to print % we should use ‘%%’ as shown below.

1printf(“there are 90%% chances of rain tonight”);

Q38. Write a code to print the following pattern.

1
12
123
1234
12345

Ans: To print the above pattern, the following code can be used.

```
1  #include<stdio.h>
2  int main()
3  {
4       for(i=1;i&amp;lt;=5;1++)
5       {
6            for(j=1;j&amp;lt;=5;j++)
7            {
8                 print("%d",j);
9            }
10           printf("n");
11      }
12      return 0;
13}
```

Q38. What is a pointer in C and why is it used?

Answer: A pointer in C is a special variable that stores the memory address of another variable instead of storing a value directly.

Pointers are used because they allow:

Direct access and modification of memory locations

Efficient array and string handling

Q39.Suppose a global variable and local variable have the same name. Is it is possible to access a global variable from a block where local variables are defined?

Ans: No. It is not possible in C. It is always the most local variable that gets preference.

What is the difference between r+ and w+ modes in C file handling?

Answer: r+ (read + write mode):

Opens an existing file for both reading and writing.

The file must already exist.

The file pointer starts at the beginning without deleting existing content.

w+ (write + read mode):

Creates a new file or overwrites an existing file.

Used for both reading and writing.

If the file exists, its previous content is deleted.

PART – B

Q1. What is Bubble Sort Explain with a program?

Ans: Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

```
1  int main()
2  {
3       int array[100], n, i, j, swap;
4       printf("Enter number of elementsn");
5       scanf("%d", &amp;amp;n);
6       printf("Enter %d Numbers:n", n);
7       for(i = 0; i&amp;lt;n; i++)
8            scanf("%d", &amp;amp;array[i]);
9            for(i = 0 ; i&amp;lt;n - 1; i++)
10           {
11                for(j = 0 ; j &amp;lt; n-i-1; j++) { if(array[j]&amp;gt;array[j+1])
12                     {
13                          swap=array[j];
14                          array[j]=array[j+1];
15                          array[j+1]=swap;
16                     }
17                }
18           }
19           printf("Sorted Array:n");
20           for(i = 0; i &amp;lt; n; i++)
21                printf("%dn", array[i]);
22           return 0;
23}
```

Q2. Develop a C program to print the following Series or Fibonacci series.

1 1 2 3 5 8.....n

Ans: #include<stdio.h>

void main()

```
{
   int t1=0,t2=1,t=0, i=1,n;
   printf("No of terms ");
   scanf("%d",&n); while(i<=n)
   {
      printf("%d\n",t2);t =
      t1 + t2;
      t1 = t2;t2=t;
      i++;
   }
}
```

Q 3. Develop a C program to print the following pyramid..

```
   1
  1 21
12 321
```

Ans:

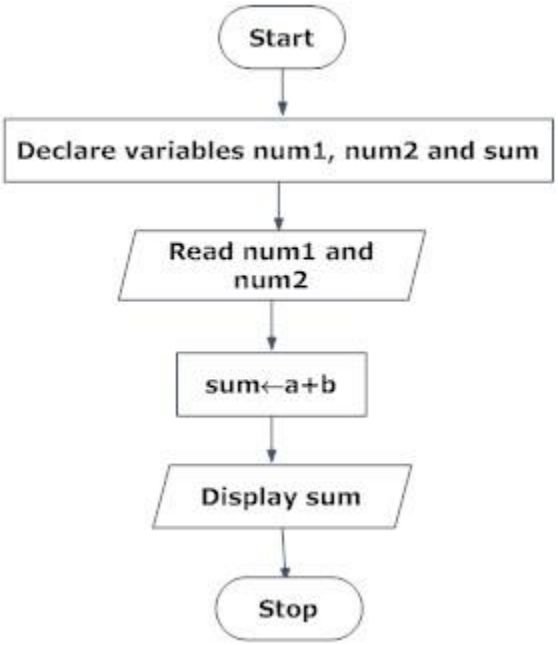
```
#include<stdio.h>void
main()
{
   int i,j,s,n;
   printf("No of terms ");
   scanf("%d",&n);
   for(i=1;i<=n;i++)
   {
      for(s=1;s<=n-i;i++)
      {
        printf("  ");
      }
```

```
    for(j=1;j<=i;j++)
    {
      printf("%d ",j);
    }
    for(j=i-1;j>=1;j--)
    {
      printf("%d ",j);
    }
     printf("\n");
}
```

Q.4 Write a program of adds two numbers and explain this program step with the help of flow chart.

Answer:

(i)Draw a flowchart to add two numbers entered by user.



Q.5 What is pointer? Discuss advantages of pointer?

Answer:

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before you can use it to store any variable address. The general form of a pointer variable declaration is:

type *var-name;

Here, type is the pointer's base type; it must be a valid C data type and var-name is the name of the pointer variable. The asterisk * you used to declare a pointer is the same asterisk that you use for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Following are the valid pointer declaration:

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

```
#include <stdio.h>
int main ()
{
  int  var1;
  char var2[10];
printf("Address of var1 variable: %x\n", &var1  );
  printf("Address of var2 variable: %x\n", &var2  );
 return 0;
}
```

Benefits(use) of pointers in c:

- Pointers provide direct access to memory
- Pointers provide a way to return more than one value to the functions
- Reduces the storage space and complexity of the program
- Reduces the execution time of the program
- Provides an alternate way to access array elements
- Pointers can be used to pass information back and forth between the calling function and called function.
- Pointers allows us to perform dynamic memory allocation and deallocation.

- Pointers helps us to build complex data structures like linked list, stack, queues, trees, graphs etc.
- Pointers allows us to resize the dynamically allocated memory block.
- Addresses of objects can be extracted using pointers

Q.6 Write a program for swapping of two number using pointer and function.

Swap Two Numbers / Variables using Pointer :

```
#include<stdio.h>
void swap( int *num1, int *num2)
{
   int temp ;
   temp  = *num1 ;
   *num1 = *num2 ;
   *num2 = temp ;
}
int main( )
{
   int num1,num2;
printf("nEnter the first number : ");
   scanf("%d",&num1);
   printf("nEnter the Second number : ");
   scanf("%d",&num2);
   swap(&num1,&num2) ;
   printf("nFirst number  : %d",num1);
   printf("nSecond number : %d",num2);
  return(0);
}
```

Q.7 Write a program to enter a number and check it is prime or not.

```
#include<stdio.h>
int main(){
   int num,i,count=0;
   printf("Enter a number: ");
   scanf("%d",&num);
   for(i=2;i<=num/2;i++){
```

```
      if(num%i==0){
       count++;
          break;
      }
   }
  if(count==0 && num!= 1)
      printf("%d is a prime number",num);
  else
    printf("%d is not a prime number",num);
  return 0;
}
```

Q.8Write a program to enter a number and check it is palindrome or not.

```
#include <stdio.h>
int main()
{
  int n, reverse=0, rem,temp;
  printf("Enter an integer: ");
  scanf("%d", &n);
  temp=n;
  while(temp!=0)
  {
    rem=temp%10;
    reverse=reverse*10+rem;
    temp/=10;
  }
/* Checking if number entered by user and it's reverse number is equal. */
  if(reverse==n)
      printf("%d is a palindrome.",n);
  else
      printf("%d is not a palindrome.",n);
  return 0;
}
```

Q .9 Write a program to enter a number and check it is Armstrong or not.

```
#include<stdio.h>
#include<conio.h>
void main()
{
```

```
int a,b,c,d;
clrscr();
printf(" Enter a number");
scanf("%d" ,&a);
d=a;
c=0;
while(a>0)
{
b=a%10;
c=c+(b*b*b);
a=a/10;
}
if(c==d)
{
printf(" Number is Armstrong");
}
else
{
printf(" Number is not Armstrong");
}
getch();
}
```

Q.10 Write a program to find minimum and maximum number from array.

Answer:

```
#include<stdio.h>
int main(){
  int a[50],size,i,big,small;

  printf("\nEnter the size of the array: ");
  scanf("%d",&size);
  printf("\nEnter %d elements in to the array: ", size);
  for(i=0;i<size;i++)
     scanf("%d",&a[i]);
```

```
  big=a[0];
  for(i=1;i<size;i++){
     if(big<a[i])
         big=a[i];
  }
  printf("Largest element: %d",big);

  small=a[0];
  for(i=1;i<size;i++){
     if(small>a[i])
         small=a[i];
  }
  printf("Smallest element: %d",small);

  return 0;
}
```

Q.11 Write a program to multiplication of two matrixes.

```
#include<stdio.h>
int main(){
  int a[5][5],b[5][5],c[5][5],i,j,k,sum=0,m,n,o,p;
  printf("\nEnter the row and column of first matrix");
  scanf("%d %d",&m,&n);
  printf("\nEnter the row and column of second matrix");
  scanf("%d %d",&o,&p);
  if(n!=o){
     printf("Matrix mutiplication is not possible");
     printf("\nColumn of first matrix must be same as row of second matrix");
  }
  else{
     printf("\nEnter the First matrix->");
     for(i=0;i<m;i++)
     for(j=0;j<n;j++)
         scanf("%d",&a[i][j]);
     printf("\nEnter the Second matrix->");
```

```
    for(i=0;i<o;i++)
    for(j=0;j<p;j++)
        scanf("%d",&b[i][j]);
    printf("\nThe First matrix is\n");
    for(i=0;i<m;i++){
    printf("\n");
    for(j=0;j<n;j++){
        printf("%d\t",a[i][j]);
    }
    }
    printf("\nThe Second matrix is\n");
    for(i=0;i<o;i++){
    printf("\n");
    for(j=0;j<p;j++){
        printf("%d\t",b[i][j]);
    }
    }
    for(i=0;i<m;i++)
    for(j=0;j<p;j++)
        c[i][j]=0;
    for(i=0;i<m;i++){ //row of first matrix
    for(j=0;j<p;j++){  //column of second matrix
        sum=0;
        for(k=0;k<n;k++)
            sum=sum+a[i][k]*b[k][j];
        c[i][j]=sum;
    }
    }
}
printf("\nThe multiplication of two matrix is\n");
for(i=0;i<m;i++){
    printf("\n");
    for(j=0;j<p;j++){
        printf("%d\t",c[i][j]);
    }
}
return 0;
```

}

Q.12Write a program for searching an element in array.

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[30],x,n,i;
/*
 a - for storing of data
 x - element to be searched
 n - no of elements in the array
 i - scanning of the array
*/
 printf("nEnter no of elements :");
scanf("%d",&n);
/* Reading values into Array */

printf("nEnter the values :");
for(i=0;i < n;i++)
scanf("%d",&a[i]);

/* read the element to be searched */

printf("nEnter the elements to be searched");
scanf("%d",&x);

/* search the element */
```

```
i=0; /* search starts from the zeroth location */
while(i < n && x!=a[i])
i++;

/* search until the element is not found i.e. x!=a[i]
search until the element could still be found i.e. i 〈 n */

if(i < n) /* Element is found */
printf("found at the location =%d",i+1);
else
printf("n not found");

getch();
}
```

Q.13 Write a program for sorting elements of array.

```
#include<stdio.h>
#include<conio.h>
voidmain()
{
int arr[10];
int i,j,temp;
clrscr();
for(i=0;i<10;i++)
scanf("%d",&arr[i]);
for(i=0;i<9;i++)
{
 for(j=i+1;j<10;j++)
 {
  if(arr[i]<arr[j])
  {
  temp=arr[i];
```

```
  arr[i]=arr[j];
  arr[j]=temp;
  }
}
}
printf("\nSortedarrayelements:\n");
for(i=0;i<10;i++)
printf("%d",arr[i]) ;
getch();
}
```

Q.14 Write a program to enter a string and check it is palindrome or not.

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
  char st[20];
  int flag=0,i,l;
  clrscr();
  printf("enter any string\n");
  gets(st);
  l=strlen(st);
  for(i=0;i<l/2;i++)
     if(st[i]!=st[l-1-i])
     {
          flag=1;
          break;
     }
  if(flag==0)
    printf("String is Palindrom");
  else
    printf("String is not palindrom\n");
getch();
}
```

Q.15 Write a program to enter a string and convert it into different case letter upper to lower or lower to upper.

```
#include<stdio.h>
#include<conio.h>
```

```
void strlwr(char[]);
void main()
{
char a[80];
clrscr();
printf("Enter any string\n");
gets(a);
printf("String in lower case\n");
strlwr(a);
getch();
}
voidstrlwr(chara[])
{

Inti;
for(i=0;a[i];i++)

if(a[i]>='A'&&a[i]<='Z')
 a[i]=a[i]+32;
 else
 a[i]=a[i];
printf("%s",a);
}
```

PART C

Q.1 What is Recursion? Write a program to enter a number and find its factorial using recursion. Also write a program for Fibonacci using recursion.

Answer:

Recursion is the process of repeating items in a self-similar way. Same applies in programming languages as well where if a programming allows you to call a function inside the same function that is called recursive call of the function as follows.

```
void recursion()
{
  recursion(); /* function calls itself */
}
int main()
{
  recursion();
}
```

The C programming language supports recursion, i.e., a function to call itself. But while using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go in infinite loop.

Recursive function are very useful to solve many mathematical problems like to calculate factorial of a number, generating Fibonacci series, etc.

Number Factorial

Following is an example, which calculates factorial for a given number using a recursive function:

```
#include <stdio.h>

int factorial(unsigned int i)
{
  if(i <= 1)
  {
    return 1;
  }
  return i * factorial(i - 1);
}
int  main()
{
   int i = 15;
   printf("Factorial of %d is %d\n", i, factorial(i));
   return 0;
}
```

```
/*c program to generate Fibonacci series using recursion method*/
#include<stdio.h>
#include<conio.h>
int fibo(int , int ); /*declaration function*/
int main()
{
 int num=1,previous_num=0;
 printf("Fibonacci Series first 30 elements: ");
 printf("\n\n\t1");
 fibo(previous_num,num); /*calling function*/
 getch();
 return 0;
}
fibo(int prev, int n) /*definition of function*/
{
 static int r=1;
 int series;
 if(r!=30)
 {
  series=prev+n;
  prev=n;
  n=series;
  printf("\n\t%d",series);
  r++;
  fibo(prev,n);
 }
}
```

Q.2 Explain different categories of function with an example. And explain getch(),getche(),puts(),gets(),strcpy() and strcat().

Answer:

For better understanding of arguments and return type in functions, user-defined functions can be categorised as:

1. Function with no arguments and no return value
2. Function with no arguments and return value
3. Function with arguments but no return value
4. Function with arguments and return value.

Let's take an example to find whether a number is prime or not using above 4 cateogories of user defined functions.

Function with no arguments and no return value.

/*C program to check whether a number entered by user is prime or not using function with no arguments and no return value*/

```
#include <stdio.h>
```

```
void prime();
int main(){
   prime();      //No argument is passed to prime().
   return 0;
}
void prime(){
/* There is no return value to calling function main(). Hence, return type of prime() is
void */
   int num,i,flag=0;
   printf("Enter positive integer enter to check:\n");
   scanf("%d",&num);
   for(i=2;i<=num/2;++i){
      if(num%i==0){
          flag=1;
       }
   }
   if (flag==1)
      printf("%d is not prime",num);
   else
     printf("%d is prime",num);
   }
```

Function prime() is used for asking user a input, check for whether it is prime of not and display it accordingly. No argument is passed and returned form prime() function.

Function with no arguments but return value

```
/*C program to check whether a number entered by user is prime or not using function
with no arguments but having return value */
#include <stdio.h>
int input();
int main(){
   int num,i,flag = 0;
   num=input();     /* No argument is passed to input() */
   for(i=2; i<=num/2; ++i){
   if(num%i==0){
      flag = 1;
      break;
   }
   }
   if(flag == 1)
      printf("%d is not prime",num);
   else
      printf("%d is prime", num);
   return 0;
}
```

```
int input(){   /* Integer value is returned from input() to calling function */
   int n;
   printf("Enter positive integer to check:\n");
   scanf("%d",&n);
   return n;
}
```

There is no argument passed to input() function But, the value of n is returned from input() to main()function.

Function with arguments and no return value

```
/*Program to check whether a number entered by user is prime or not using function with arguments and no return value */
#include <stdio.h>
void check_display(int n);
int main(){
   int num;
   printf("Enter positive enter to check:\n");
   scanf("%d",&num);
   check_display(num);  /* Argument num is passed to function. */
   return 0;
}
void check_display(int n){
/* There is no return value to calling function. Hence, return type of function is void. */
   int i, flag = 0;
for(i=2; i<=n/2; ++i){
   if(n%i==0){
      flag = 1;
      break;
   }
   }
   if(flag == 1)
      printf("%d is not prime",n);
   else
      printf("%d is prime", n);
}
```

Here, check_display() function is used for check whether it is prime or not and display it accordingly. Here, argument is passed to user-defined function but, value is not returned from it to calling function.

Function with argument and a return value

```
/* Program to check whether a number entered by user is prime or not using function with argument and return value */
#include <stdio.h>
int check(int n);
```

```
int main(){
   int num,num_check=0;
   printf("Enter positive enter to check:\n");
   scanf("%d",&num);
   num_check=check(num); /* Argument num is passed to check() function. */
   if(num_check==1)
     printf("%d is not prime",num);
   else
     printf("%d is prime",num);
   return 0;
}
int check(int n){
/* Integer value is returned from function check() */
   int i;
   for(i=2;i<=n/2;++i){
   if(n%i==0)
      return 1;
}
  return 0;
}
```

Here, check() function is used for checking whether a number is prime or not. In this program, input from user is passed to function check() and integer value is returned from it. If input the number is prime, 0 is returned and if number is not prime, 1 is returned.

Use of getch(),getche() and getchar() in C

Function : getchar()
getchar() is used to get or read the input (i.e a single character) at run time.
Declaration:
int getchar(void);
Example Declaration:
char ch;
ch = getchar();

Return Value:
This function return the character read from the keyboard.
Example Program:

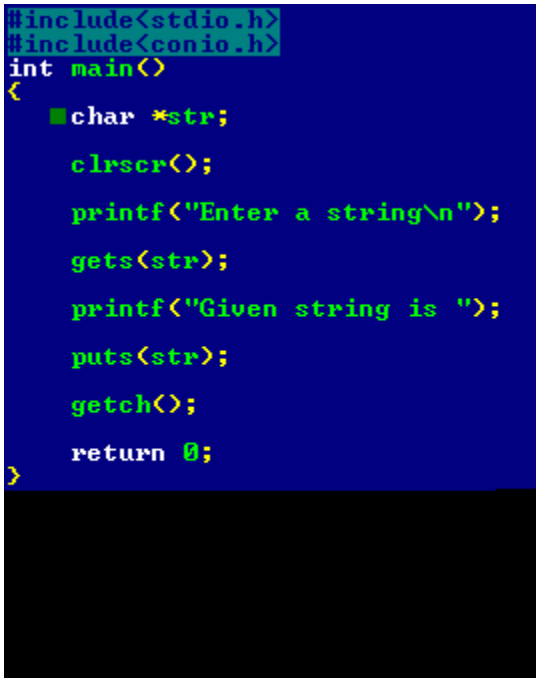
```
void main()
{
char ch;
ch = getchar();
printf("Input Char Is :%c",ch);
```

}
Program Explanation:
Here,declare the variable ch as char data type, and then get a value through getchar()library function and store it in the variable ch.And then,print the value of variable ch.
During the program execution, a single character is get or read through the getchar(). The given value is displayed on the screen and the compiler wait for another character to be typed. If you press the enter key/any other characters and then only the given character is printed through the printf function.
Function : getche()
getche() is used to get a character from console, and echoes to the screen.
Declaration:
int getche(void);
Example Declaration:
char ch;
ch = getche();
Remarks:
getche reads a single character from the keyboard and echoes it to the current text window, using direct video or BIOS.
Return Value:
This function return the character read from the keyboard.
Example Program:
```
void main()
{
char ch;
ch = getche();
printf("Input Char Is :%c",ch);
}
```
Program Explanation:
Here,declare the variable ch as char data type, and then get a value through getche()library function and store it in the variable ch.And then,print the value of variable ch.During the program execution, a single character is get or read through the getche(). The given value is displayed on the screen and the compiler does not wait for another character to be typed. Then,after wards the character is printed through the printf function.
Function : getch()
getch() is used to get a character from console but does not echo to the screen.
Declaration:
int getch(void);
Example Declaration:
char ch;
ch = getch(); (or) getch();
Remarks:

getch reads a single character directly from the keyboard, without echoing to the screen.
Return Value:
This function return the character read from the keyboard.
Example Program:

```
void main()
{
char ch;
ch = getch();
printf("Input Char Is :%c",ch);
}
```

It is used to display a string on a standard output device. The puts() function automatically inserts a newline character at the end of each string it displays, so each subsequent string displayed with puts() is on its own line. puts() returns non-negative on success, or EOF on failure.
The following program illustrates the use of puts function.



```
#include<stdio.h>
#include<conio.h>
int main()
{
   char *str;
    clrscr();
    printf("Enter a string\n");
    gets(str);
    printf("Given string is ");
    puts(str);
    getch();
    return 0;
}
```

strcat(s1, s2);
Concatenates string s2 onto the end of string s1.
strcpy(s1, s2);
Copies string s2 into string s1.

```
#include <stdio.h>
#include <string.h>
int main ()
{
  char str1[12] = "Hello";
  char str2[12] = "World";
  char str3[12];
```

```
   int  len ;
/* copy str1 into str3 */
   strcpy(str3, str1);
   printf("strcpy( str3, str1) :  %s\n", str3 );
 /* concatenates str1 and str2 */
   strcat( str1, str2);
   printf("strcat( str1, str2):   %s\n", str1 );
 /* total lenghth of str1 after concatenation */
   len = strlen(str1);
   printf("strlen(str1) :  %d\n", len );
 return 0;
}
```

Q.3 What is nested structure? There is a structure called employee that hold information like employee id, name and date of joining. Write a program to create an array of structure and enter some date into it. Then ask the user to enter name. Display the record of those employee whose name is match to the enter name.

Answer:

Structures can be used as structures within structures. It is also called as 'nesting of structures'.

```
struct date
{
int date;
int month;
int year;
};

struct Employee
   {
char ename[20];
int ssn;
float salary;
struct date doj;
}emp1;
```

```
Accessing Month Field :  emp1.doj.month
Accessing day Field   :  emp1.doj.day
Accessing year Field  :  emp1.doj.year
```

Example:

```
#include <stdio.h>
```

```
struct Employee
{
char ename[20];
int ssn;
float salary;
struct date
      {
int date;
int month;
int year;
      }doj;
}emp = {"Pritesh",1000,1000.50,{22,6,1990}};

int main(int argc, char *argv[])
{
printf("\nEmployee Name   : %s",emp.ename);
printf("\nEmployee SSN    : %d",emp.ssn);
printf("\nEmployee Salary : %f",emp.salary);
printf("\nEmployee DOJ    : %d/%d/%d", \
        emp.doj.date,emp.doj.month,emp.doj.year);
return 0;
}
```

Q.4 (i)Write a program to write and read record of employee using fread() and fwrite() function.

(ii) Write a program to write and read record of student using fscanf() and fprintf() function.

(iii) write a program to open a pre existing file and add information at the end of the file. Display the contents of the file before and after appending.

Answer:

(i)

```
/* program to read & write structure data from existing file
   using fread() & fwrite() functions */
#include<stdio.h>
#include<conio.h>
 struct emp
 {char name[20];
  int id;
  }e;
 void main()
 { char ch='y';
   FILE *pk=fopen("keviv.txt","aw+");
   clrscr();
    if(pk==NULL)
```

```
    { printf("\n\n FILE DOES NOT EXIST");
      getch();
      exit(0);
    }
   while(ch=='y')
   { printf("\n\n Enter records of employee:\n");
     printf("\n Enter name: ");
     scanf("%s",e.name);
     printf("\n Enter ID: ");
     scanf("%d",&e.id);
     fwrite(&e,sizeof(e),1,pk);
     printf("\n\n Do u want to continue... ");
     fflush(stdin);
     scanf("%c",&ch);
   }
   fclose(pk);
   pk=fopen("keviv.txt","ar+");
   while(fread(&e,sizeof(e),1,pk)!=EOF)
   { printf(" %s %d\n",e.name,e.id);
     break;
   }
   fclose(pk);
   getch();
 }
```

(ii)

```
Void main(int argc,char* argv[]) /* command line arguments */
 {
   FILE *ft,*fs; /* file pointers declaration*/
   Int c=0;
   If(argc!=3)
       Printf("\n insufficient arguments");
/* opening files*?
   Fs=fopen(argv[1],"r");
   Ft=fopen(argv[2],"w");
/*error handling*/
 If (fs==NULL)
   {
     Printf("\n source file opening error");
     Exit(1) ;
   }
If (ft==NULL)
   {
     Printf("\n target file opening error");
```

```
    Exit(1) ;
  }
/*Reading file and writing into second file*/
 While(!feof(fs))
  {
       Fputc(fgetc(fs),ft);
       C++;
   }
Printf("\n bytes copied from %s file to %s file =%d",argv[1].argv[2],c);
C=Fcloseall(); /*closing all files*/
Printf("files closed=%d",c);
}
```

(iii)

```
 Void main( )
 {
  FILE *ft; /* file pointers declaration*/
  char c;
printf("displaying the contents of the file before appending");
   Ft=fopen("ram.txt","r");
If (ft==NULL)
  {
   Printf("\n target file opening error");
   Exit(1) ;
  }
While(!feof(fs))
 {
   C=fgetc(ft);
   Printf("%c",c);
   }
fclose(ft);
/*opening the file for appending data*/
   Ft=fopen("ram.txt","a");
/*appending data*/
While(c!='.')
 {
 C=getche();
 Fputc(c,ft);
}
Fclose(ft);
Printf(" displaying the contents of the file after appending */
   Ft=fopen("ram.txt","r");
While(!feof(fs))
  {
```

```
    C=fgetc(ft);
    Printf("%c",c);
    }
fclose(ft);
}
```

Q.5 Short notes on:
(i) fseek() (ii) ftell() (iii)rewind() (iv) feof()

Answer:

(i)

The C library function int fseek(FILE *stream, long int offset, int whence) sets the file position of thestream to the given offset.

Declaration

Following is the declaration for fseek() function.

int fseek(FILE *stream, long int offset, int whence)

Parameters

- stream -- This is the pointer to a FILE object that identifies the stream.
- offset -- This is the number of bytes to offset from whence.
- whence -- This is the position from where offset is added. It is specified by one of the following constants:

Constant	Description
SEEK_SET	Beginning of file
SEEK_CUR	Current position of the file pointer
SEEK_END	End of file

Return Value

This function returns zero if successful, else it returns nonzero value.

Example

The following example shows the usage of fseek() function.

```
#include <stdio.h>

int main ()
{
  FILE *fp;

 fp = fopen("file.txt","w+");
  fputs("This is tutorialspoint.com", fp);
   fseek( fp, 7, SEEK_SET );
  fputs(" C Programming Langauge", fp);
  fclose(fp);
    return(0);
}
```

(ii)

The C library function long int ftell(FILE *stream) returns the current file position of the given stream.

Declaration

Following is the declaration for ftell() function.

long int ftell(FILE *stream)

Parameters

- **stream** -- This is the pointer to a FILE object that identifies the stream.

Return Value This function returns the current value of the position indicator. If an error occurs, -1L is returned, and the global variable errno is set to a positive value.

Example

The following example shows the usage of ftell() function.

```
#include <stdio.h>
int main ()
{
  FILE *fp;
  int len;
 fp = fopen("file.txt", "r");
  if( fp == NULL )
  {
    perror ("Error opening file");
    return(-1);
  }
  fseek(fp, 0, SEEK_END);
 len = ftell(fp);
  fclose(fp);
printf("Total size of file.txt = %d bytes\n", len);
   return(0);
}
```

(i) The C library function void rewind(FILE *stream) sets the file position to the beginning of the file of the given stream.

Declaration

Following is the declaration for rewind() function.

void rewind(FILE *stream)

Parameters

- **stream** -- This is the pointer to a FILE object that identifies the stream.

Return Value This function does not return any value.

(ii) The C library function int feof(FILE *stream) tests the end-of-file indicator for the given stream.

s

Following is the declaration for feof() function.

int feof(FILE *stream)

Parameters

- stream -- This is the pointer to a FILE object that identifies the stream.

Return Value
This function returns a non-zero value when End-of-File indicator associated with the stream is set, else zero is returned.
Example
The following example shows the usage of feof() function.

```
#include <stdio.h>
int main ()
{
  FILE *fp;
  int c;

  fp = fopen("file.txt","r");
  if(fp == NULL)
  {
    perror("Error in opening file");
    return(-1);
  }
  while(1)
  {
    c = fgetc(fp);
    if( feof(fp) )
    {
       break ;
    }
    printf("%c", c);
  }
  fclose(fp);
  return(0);
}
```

Q.6 (i)Write a program to enter employee record using pointer to structure.
(ii) Write a program to enter student record using structure and function.
Answer:
(i)
Pointers can be accessed along with structures. A pointer variable of structure can be created as below:

```
struct name {
   member1;
   member2;
   .
   .
};
-------- Inside function -------
```

struct name *ptr;
Here, the pointer variable of type struct name is created.

Structure's member through pointer can be used in two ways:

Referencing pointer to another address to access memory
Using dynamic memory allocation
Consider an example to access structure's member through pointer.

```
#include <stdio.h>
struct name{
  int a;
  float b;
};
int main(){
   struct name *ptr,p;
   ptr=&p;          /* Referencing pointer to memory address of p */
   printf("Enter integer: ");
   scanf("%d",&(*ptr).a);
   printf("Enter number: ");
   scanf("%f",&(*ptr).b);
   printf("Displaying: ");
   printf("%d%f",(*ptr).a,(*ptr).b);
   return 0;
}
```

In this example, the pointer variable of type struct name is referenced to the address of p. Then, only the structure member through pointer can can accessed.

Structure pointer member can also be accessed using -> operator.

(*ptr).a is same as ptr->a
(*ptr).b is same as ptr->b
Accessing structure member through pointer using dynamic memory allocation
To access structure member using pointers, memory can be allocated dynamically using malloc() function defined under "stdlib.h" header file.

Syntax to use malloc()
ptr=(cast-type*)malloc(byte-size)

```
#include <stdio.h>
struct employee
{
   char firstname[40];
```

```
   char lastname[40];
   int id;
};
typedef struct employee Employee[3];

/* Input the employee data interactively from the keyboard */
void InputEmployeeRecord(Employee *ptrEmployee);

void main()
{
   Employee e1;
   InputEmployeeRecord(&e1);
}

void InputEmployeeRecord(Employee *ptrEmployee)
{
   int i=0;
   for(i=0;i<3;i++)
   {
      printf("Enter the employee ID")
      scanf("%d", Employee.id);
      printf("Enter the employee first name")
      gets(firstname);
      printf("Enter the employee last name")
      gets(lastname);
   }
}
```

(ii)

Passing structure to function in C:

It can be done in below 3 ways.

Passing structure to a function by valuePassing structure to a function byaddress(reference)No need to pass a structure – Declare structure variable as globalExample program – passing structure to function in C by value:

In this program, the whole structure is passed to another function by value. It means the whole structure is passed to another function with all members and their values. So, this structure can be accessed from called function. This concept is very useful while writing very big programs in C.

```
#include <stdio.h>
#include <string.h>
```

```
struct student
{
        int id;
        char name[20];
        float percentage;
};

void func(struct student record);

int main()
{
        struct student record;

        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        func(record);
        return 0;
}

void func(struct student record)
{
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
}
```

Output:

```
Id is: 1
Name is: Raju
Percentage is: 86.500000
```

Example program – Passing structure to function in C by address:

In this program, the whole structure is passed to another function by address. It means only the address of the structure is passed to another function. The whole structure is not passed to another function with all members and their values. So, this structure can be accessed from called function by its address.

```
#include <stdio.h>
#include <string.h>
```

```
struct student
{
        int id;
        char name[20];
        float percentage;
};

void func(struct student *record);

int main()
{
        struct student record;

        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        func(&record);
        return 0;
}

void func(struct student *record)
{
        printf(" Id is: %d \n", record->id);
        printf(" Name is: %s \n", record->name);
        printf(" Percentage is: %f \n", record->percentage);
}
```

Output:

```
Id is: 1
Name is: Raju
Percentage is: 86.500000
```

Example program to declare a structure variable as global in C:

Structure variables also can be declared as global variables as we declare other variables in C. So, When a structure variable is declared as global, then it is visible to all the functions in a program. In this scenario, we don't need to pass the structure to any function separately.

```
#include <stdio.h>
#include <string.h>

struct student
```

```
{
        int id;
        char name[20];
        float percentage;
};
struct student record; // Global declaration of structure

void structure_demo();

int main()
{
        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        structure_demo();
        return 0;
}

void structure_demo()
{
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
}
```

Output:

```
Id is: 1
Name is: Raju
Percentage is: 86.500000
```

Q7 Write a program to copy the content of one file to another file.

Answer:

```
#include<stdio.h>
#include<conio.h>
void main()
{
FILE *fp1,*fp2;
char ch,fname1[20],fname2[20];
printf("\n enter sourse file name");
gets(fname1);
printf("\n enter sourse file name");
gets(fname2);
fp1=fopen(fname1,"r");
```

```
fp2=fopen(fname2,"w");
if(fp1==NULL||fp2==NULL)
{
printf("unable to open");
exit(0);
}
do
{
ch=fgetc(fp1);
fputc(ch,fp2);
}
while(ch!=EOF);
fcloseall();
getch();
}
```

Q 8. Describe Pointer to function and pointer and function with example .

Pointers to Functions

You can even create pointers that point to functions. The primary use of such pointers is to pass functions as arguments to another function. You have to be very careful while declaring pointers to functions. Take note of the position of the parentheses:

return-data (*pointer-name) (arguments);

We'll write a simple program to add two numbers. For this purpose we shall define a function called sum( ). This function sum ( ), will be passed to another function called func( ) which will simply call the sum ( ) function.

```
int sum(inta,intb)
{
return(a+b);
}
Void func(int d,Int e,int (*p1) (int,int))
{
printf("d",*p1)(d,e));
}
Int main()
{
int(*p)(int,int);
p=sum;
func(5,6,p);
return0;
}
```

The output is:

The result is : 11

Though the program is simple a few statements might appear confusing. The statement int (*p) (int,int);declares a pointer to a function that has a return value of integer and takes two arguments of type integer.
sum( ) is a function with return data type of integer and also with two arguments of type integer. Hence the pointer 'p' can point to the function sum ( ). Thus we assign the address of the function sum ( ) to 'p':p = sum;We've also defined another function called as 'func ( )' which takes two integer arguments and a third argument which is a pointer to a function. The idea is to pass the function sum ( ) to the function 'func( )' and call the sum ( ) function from func ( ).
func(5,6,p);will call the func ( ) function and it will pass pointer 'p' to func ( ). Remember that p is a pointer to sum ( ). Hence in reality we are actually passing a function as argument to another function.
Let's expand the program one step further by creating another function called product( ) which will return the product of the two arguments.

```
#include<iostream.h>
intsum(inta,intb)
{
    return(a+b);
}
intproduct(intx,inty)
{
    return(x*y);
}
voidfunc(intd,inte,int(*p1)(int,int))
{
    cout<<endl<<"Theresultis:"<<(*p1)(d,e);
}
intmain()
{
    int(*p)(int,int);
    p=sum;
    func(5,6,p);
    p=product;
    func(5,6,p);
    return0;
}
```

The output is:
The result is : 11
The result is : 30
The program is very similar to the previous one. I just wanted to illustrate that you can pass any function to func ( ) as long as it has a return type of integer and it has two integer arguments.

Q.9Describe the following?

(a)Enumerated data type

(b) Operators

(a) Enumeration type allows programmer to define their own data type .

Keyword enum is used to defined enumerated data type.

enum type_name{ value1, value2,...,valueN };

Here, type_name is the name of enumerated data type or tag.

And value1, value2,....,valueN are values of type type_name.

By default, value1 will be equal to 0, value2 will be 1 and so on but, the programmer can change the default value as below:

```
enum suit{
   club=0;
   diamonds=10;
   hearts=20;
   spades=3;
};
```

Declaration of enumerated variable

Above code defines the type of the data but, no any variable is created. Variable of type enum can be created as:

```
enum boolean{
   false;
   true;
};
enum boolean check;
```

Here, a variable check is declared which is of type enum boolean.

Example of enumerated type

```
#include <stdio.h>
enum week{ sunday, monday, tuesday, wednesday, thursday, friday, saturday};
int main(){
   enum week today;
   today=wednesday;
   printf("%d day",today+1);
   return 0;
  }
```

Output

4 day

(b) Operators

What is Operator? Simple answer can be given using expression 4 + 5 is equal to 9. Here 4 and 5 are called operands and + is called operator. C language supports following type of operators.

- Arithmetic Operators
- Logical (or Relational) Operators
- Bitwise Operators
- Assignment Operators
- Misc Operators

Lets have a look on all operators one by one.

Arithmetic Operators:

There are following arithmetic operators supported by C language:

Assume variable A holds 10 and variable B holds 20 then:

Show Examples

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiply both operands	A * B will give 200
/	Divide numerator by denumerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Logical (or Relational) Operators:

There are following logical operators supported by C language

Assume variable A holds 10 and variable B holds 20 then:

Show Examples

Operator	Description	Example
==	Checks if the value of two operands is equal or not, if yes then condition	(A == B) is not true.

	becomes true.	
!=	Checks if the value of two operands is equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.
&&	Called Logical AND operator. If both the operands are non zero then then condition becomes true.	(A && B) is true.
||	Called Logical OR Operator. If any of the two operands is non zero then then condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is false.

Bitwise Operators:

Bitwise operator works on bits and perform bit by bit operation.

Assume if A = 60; and B = 13; Now in binary format they will be as follows:

A = 0011 1100
B = 0000 1101

A&B = 0000 1100
A|B = 0011 1101
A^B = 0011 0001
~A = 1100 0011
Show Examples
There are following Bitwise operators supported by C language

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
|	Binary OR Operator copies a bit if it exists in eather operand.	(A | B) will give 61 which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~	Binary Ones Complement Operator is unary and has the efect of 'flipping' bits.	(~A) will give -60 which is 1100 0011
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 0000 1111

Assignment Operators:
There are following assignment operators supported by C language:
Show Examples

Operator	Description	Example
=	Simple assignment operator,	C = A + B will assigne value of A + B into

	Assigns values from right side operands to left side operand	C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator	C ^= 2 is same as C = C ^ 2
|=	bitwise inclusive OR and	C |= 2 is same as C = C | 2

	assignment operator	

Short Notes on L-VALUE and R-VALUE:

x = 1; takes the value on the right (e.g. 1) and puts it in the memory referenced by x. Here x and 1 are known as L-VALUES and R-VALUES respectively L-values can be on either side of the assignment operator where as R-values only appear on the right.

So x is an L-value because it can appear on the left as we've just seen, or on the right like this: y = x; However, constants like 1 are R-values because 1 could appear on the right, but 1 = x; is invalid.

Misc Operators

There are few other operators supported by C Language.

Show Examples

Operator	Description	Example
sizeof()	Returns the size of an variable.	sizeof(a), where a is interger, will return 4.
&	Returns the address of an variable.	&a; will give actaul address of the variable.
*	Pointer to a variable.	*a; will pointer to a variable.
? :	Conditional Expression	If Condition is true ? Then value X : Otherwise value Y

Operators Categories:

All the operators we have discussed above can be categorised into following categories:

- Postfix operators, which follow a single operand.
- Unary prefix operators, which precede a single operand.
- Binary operators, which take two operands and perform a variety of arithmetic and logical operations.
- The conditional operator (a ternary operator), which takes three operands and evaluates either the second or third expression, depending on the evaluation of the first expression.
- Assignment operators, which assign a value to a variable.
- The comma operator, which guarantees left-to-right evaluation of comma-separated expressions.

Q.9Describe the following?

Concept of Preprocessor, Macro Substitution
TypeConversion

Storage Class

The C Preprocessor

Recall that preprocessing is the first step in the C program compilation stage -- this feature is unique to C compilers.

The preprocessor more or less provides its own language which can be a very powerful tool to the programmer. Recall that all preprocessor directives or commands begin with a #.

Use of the preprocessor is advantageous since it makes:

- programs easier to develop,
- easier to read,
- easier to modify
- C code more transportable between different machine architectures.

The preprocessor also lets us customise the language. For example to replace { ... } block statements delimiters by PASCAL like begin ... end we can do:

```
#define begin {
#define end  }
```

During compilation all occurrences of begin and end get replaced by corresponding { or } and so the subsequent C compilation stage does not know any difference!!!.

Lets look at #define in more detail

#define

Use this to define constants or any macro substitution. Use as follows:

```
#define  <macro><replacement name>
```

For Example

```
 #define FALSE  0
#define TRUE  !FALSE
```

(b)A storage class defines the scope (visibility) and life time of variables and/or functions within a C Program.

There are following storage classes which can be used in a C Program

- auto
- register
- static
- extern

auto - Storage Class

auto is the default storage class for all local variables.

```
{
 int Count;
 auto int Month;
}
```

The example above defines two variables with the same storage class. auto can only be used within functions, i.e. local variables.

register - Storage Class

register is used to define local variables that should be stored in a register instead of RAM. This means that the variable has a maximum size equal to the register size (usually one word) and cant have the unary '&' operator applied to it (as it does not have a memory location).

```
{
 register int  Miles;
}
```

Register should only be used for variables that require quick access - such as counters. It should also be noted that defining 'register' goes not mean that the variable will be stored in a register. It means that it MIGHT be stored in a register - depending on hardware and implimentation restrictions.

static - Storage Class

static is the default storage class for global variables. The two variables below (count and road) both have a static storage class.

```
   static int Count;
int Road;

{
   printf("%d\n", Road);
}
```

static variables can be 'seen' within all functions in this source file. At link time, the static variables defined here will not be seen by the object modules that are brought in.

static can also be defined within a function. If this is done the variable is initalised at run time but is not reinitalized when the function is called. This inside a function static variable retains its value during vairous calls.

extern - Storage Class

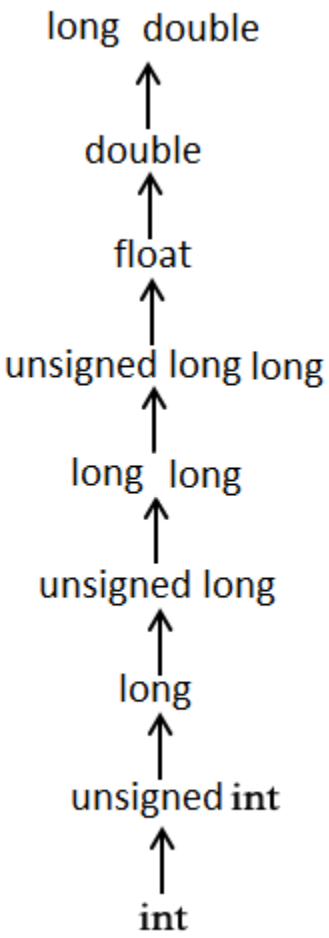
extern is used to give a reference of a global variable that is visible to ALL the program files. When you use 'extern' the variable cannot be initalized as all it does is point the variable name at a storage location that has been previously defined.
When you have multiple files and you define a global variable or function which will be used in other files also, then extern will be used in another file to give reference of defined variable or function. Just for understanding extern is used to decalre a global variable or function in another files.

Type casting is a way to convert a variable from one data type to another data type. For example, if you want to store a long value into a simple integer then you can type cast long to int. You can convert values from one type to another explicitly using the cast operator as follows:
(type_name) expression
Consider the following example where the cast operator causes the division of one integer variable by another to be performed as a floating-point operation:

```
#include <stdio.h>
main()
{
   int sum = 17, count = 5;
   double mean;
mean = (double) sum / count;
   printf("Value of mean : %f\n", mean );

}
```

When the above code is compiled and executed, it produces the following result:
Value of mean : 3.400000
It should be noted here that the cast operator has precedence over division, so the value of sum is first converted to type double and finally it gets divided by count yielding a double value.
Usual Arithmetic Conversion
The usual arithmetic conversions are implicitly performed to cast their values in a common type. Compiler first performs integer promotion, if operands still have different types then they are converted to the type that appears highest in the following hierarchy:



Q.10 Short notes on:
(a) Command line argument (b) void pointer (c) typedef, sizeof
(d)while and do while (e) dynamic memory allocation (f) constants in C

Answer:

(a)

It is possible to pass some values from the command line to your C programs when they are executed. These values are called command line arguments and many times they are important for your program specially when you want to control your program from outside instead of hard coding those values inside the code.
The command line arguments are handled using main() function arguments
where argc refers to the number of arguments passed, and argv[] is a pointer array which points to each argument passed to the program. Following is a simple example which checks if there is any argument supplied from the command line and take action accordingly:

```
#include <stdio.h>
int main( int argc, char *argv[] )
```

```
{
  if( argc == 2 )
  {
    printf("The argument supplied is %s\n", argv[1]);
  }
  else if( argc > 2 )
  {
    printf("Too many arguments supplied.\n");
  }
  else
  {
    printf("One argument expected.\n");
  }
}
```

When the above code is compiled and executed with a single argument, it produces the following result.

```
$./a.out testing
```

The argument supplied is testingWhen the above code is compiled and executed with a two arguments, it produces the following result.

```
$./a.out testing1 testing2
```

Too many arguments supplied.

When the above code is compiled and executed without passing any argument, it produces the following result.

```
$./a.out
```

One argument expectedIt should be noted that argv[0] holds the name of the program itself and argv[1] is a pointer to the first command line argument supplied, and *argv[n] is the last argument. If no arguments are supplied, argc will be one, otherwise and if you pass one argument then argc is set at 2.

You pass all the command line arguments separated by a space, but if argument itself has a space then you can pass such arguments by putting them inside double quotes "" or single quotes ''. Let us re-write above example once again where we will print program name and we also pass a command line argument by putting inside double quotes:

```
#include <stdio.h>
int main( int argc, char *argv[] )
{
  printf("Program name %s\n", argv[0]);

  if( argc == 2 )
  {
    printf("The argument supplied is %s\n", argv[1]);
  }
  else if( argc > 2 )
  {
```

```
    printf("Too many arguments supplied.\n");
  }
  else
  {
    printf("One argument expected.\n");
  }
}
```

When the above code is compiled and executed with a single argument separated by space but inside double quotes, it produces the following result.

```
$./a.out "testing1 testing2"
Programm name ./a.out
The argument supplied is testing1 testing2
```

(b)

A pointer variable declared using a particular data type can not hold the location address of variables of other data types. It is invalid and will result in a compilation error.

Ex:-
```
char  *ptr;
int  var1;
ptr=&var1; // This is invalid because 'ptr' is a character pointer variable.
```

Here comes the importance of a "void pointer". A void pointer is nothing but a pointer variable declared using the reserved word in C 'void'.

Ex:-
```
void  *ptr; // Now ptr is a general purpose pointer variable
```

When a pointer variable is declared using keyword void – it becomes a general purpose pointer variable. Address of any variable of any data type (char, int, float etc.)can be assigned to a void pointer variable.

Dereferencing a void pointer

We have seen about dereferencing a pointer variable in our article – Introduction to pointers in C. We use the indirection operator * to serve the purpose. But in the case of a void pointer we need to typecast the pointer variable to dereference it. This is because a void pointer has no data type associated with it. There is no way the compiler can know (or guess?) what type of data is pointed to by the void pointer. So to take the data pointed to by a void pointer we typecast it with the correct type of the data holded inside the void pointers location.

Example program:-

```
#include
void main()
{
int a=10;
float b=35.75;
void *ptr; // Declaring a void pointer
ptr=&a; // Assigning address of integer to void pointer.
printf("The value of integer variable is= %d",*( (int*) ptr) );// (int*)ptr - is used for type casting. Where as *((int*)ptr) dereferences the typecasted void pointer variable.
```

```
ptr=&b; // Assigning address of float to void pointer.
printf("The value of float variable is= %f",*( (float*) ptr) );
}
```

(c)

The C programming language provides a keyword called typedef, which you can use to give a type a new name. Following is an example to define a term BYTE for one-byte numbers:

typedef unsigned char BYTE;

After this type definitions, the identifier BYTE can be used as an abbreviation for the type unsigned char, for example:.

BYTE b1, b2;

By convention, uppercase letters are used for these definitions to remind the user that the type name is really a symbolic abbreviation, but you can use lowercase, as follows:

typedef unsigned char byte;

sizeof:

The sizeof operator gives the amount of storage, in bytes, required to store an object of the type of the operand. This operator allows you to avoid specifying machine-dependent data sizes in your programs.

sizeof unary-expression

sizeof (type-name)

(e)

The exact size of array is unknown untill the compile time,i.e., time when a compier compiles code written in a programming language into a executable form. The size of array you have declared initially can be sometimes insufficient and sometimes more than required. Dynamic memory allocation allows a program to obtain more memory space, while running or to release space when no space is required.

Although, C language inherently does not has any technique to allocated memory dynamically, there are 4 library functions under "stdlib.h" for dynamic memory allocation.

Function	Use of Function
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	dellocate the previously allocated space
realloc()	Change the size of previously allocated space

malloc()

The name malloc stands for "memory allocation". The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.

Syntax of malloc()

ptr=(cast-type*)malloc(byte-size)

Here, ptr is pointer of cast-type. The malloc() function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns NULL pointer.

ptr=(int*)malloc(100*sizeof(int));

This statement will allocate either 200 or 400 according to size of int 2 or 4 bytes respectively and the pointer points to the address of first byte of memory.

calloc()

The name calloc stands for "contiguous allocation". The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size and sets all bytes to zero.

Syntax of calloc()

ptr=(cast-type*)calloc(n,element-size);

This statement will allocate contiguous space in memory for an array of n elements. For example:

ptr=(float*)calloc(25,sizeof(float));

This statement allocates contiguous space in memory for an array of 25 elements each of size of float, i.e, 4 bytes.

free()

Dynamically allocated memory with either calloc() or malloc() does not get return on its own. The programmer must use free() explicitly to release space.

syntax of free()

free(ptr);

This statement cause the space in memory pointer by ptr to be deallocated.

(f)

Constants

Constants are the terms that can't be changed during the execution of a program. For example: 1, 2.5, "Programming is easy." etc. In C, constants can be classified as:

Integer constants

Integer constants are the numeric constants(constant associated with number) without any fractional part or exponential part. There are three types of integer constants in C language: decimal constant(base 10), octal constant(base 8) and hexadecimal constant(base 16) .

Decimal digits: 0 1 2 3 4 5 6 7 8 9

Octal digits: 0 1 2 3 4 5 6 7

Hexadecimal digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F.

For example:

Decimal constants: 0, -9, 22 etc

Octal constants: 021, 077, 033 etc
Hexadecimal constants: 0x7f, 0x2a, 0x521 etc
Notes:

1. You can use small caps a, b, c, d, e, f instead of uppercase letters while writing a hexadecimal constant.
2. Every octal constant starts with 0 and hexadecimal constant starts with 0x in C programming.

Floating-point constants
Floating point constants are the numeric constants that has either fractional form or exponent form. For example:
-2.0
0.0000234
-0.22E-5
Note:Here, E-5 represents 10-5. Thus, -0.22E-5 = -0.0000022.
Character constants
Character constants are the constant which use single quotation around characters. For example: 'a', 'l', 'm', 'F' etc.
Escape Sequences
Sometimes, it is necessary to use newline(enter), tab, quotation mark etc. in the program which either cannot be typed or has special meaning in C programming. In such cases, escape sequence are used. For example: \n is used for newline. The backslash(\ ) causes "escape" from the normal way the characters are interpreted by the compiler.

Escape Sequences	
Escape Sequences	Character
\b	Backspace
\f	Form feed
\n	Newline
\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark

Escape Sequences	
Escape Sequences	Character
\0	Null character

String constants

String constants are the constants which are enclosed in a pair of double-quote marks. For example:

"good" //string constant
"" //null string constant
" " //string constant of six white space
"x" //string constant having single character.
"Earth is round\n" //prints string with newline

Enumeration constants

Keyword enum is used to declare enumeration types. For example:

enum color {yellow, green, black, white};

Here, the variable name is color and yellow, green, black and white are the enumeration constants having value 0, 1, 2 and 3 respectively by default

Q.11

(a) Write a program to enter a string and count length of string and reverse string.

(b) Write a program to enter a year and check it is leap year or not.

(a)

```
#include<stdio.h>
#include<string.h>
void main()
{
char str[100],temp;
int i,j=0;
 printf("nEnter the string :");
 gets(str);
 i=0;
 j=strlen(str)-1;
while(i<j)
    {
    temp=str[i];
    str[i]=str[j];
    str[j]=temp;
    i++;
```

```
    j--;
    }
 printf("nReverse string is :%s",str);
return(0);
}
```

(b)

```
#include <stdio.h>
 int main()
{
  int year;
  printf("Enter a year to check if it is a leap year\n");
  scanf("%d", &year);
   if ( year%400 == 0)
    printf("%d is a leap year.\n", year);
  else if ( year%100 == 0)
    printf("%d is not a leap year.\n", year);
  else if ( year%4 == 0 )
    printf("%d is a leap year.\n", year);
  else
    printf("%d is not a leap year.\n", year);
  return 0;
}
```

Q.12 Write a program to Accessing an array element using pointer.

Answer:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10];
int i,sum=0;
int *ptr;
 printf("Enter 10 elements:n");
for(i=0;i<10;i++)
    scanf("%d",&a[i]);
 ptr = a;          /* a=&a[0] */
for(i=0;i<10;i++)
    {
    sum = sum + *ptr;    //*p=content pointed by 'ptr'
    ptr++;
    }
```

```
printf("The sum of array elements is %d",sum);
}
```

Q.13

(a) Describe array & pointer?

(b) What is the difference between array, structure, union and pointer.

Arrays

An array is a fixed-length collection of objects, which are stored sequentially in memory. There are only three things you can do with an array:

1. sizeof - getitssize
   You can apply sizeof to it. An array x of N elements of type T (T x[N]) has the size N* sizeof (T), which is what you should expect. For example,
   if sizeof (int) == 2and int arr[5];, then sizeof arr == 10 == 5 * 2 == 5 * sizeof (int).
2. & - getitsaddress
   You can take its address with &, which results in a pointer to the entire array.
3. anyotheruse- implicitpointerconversion
   Any other use of an array results in a pointer to the first array element (the array "decays" to a pointer).

That's all. Yes, this means arrays don't provide direct access to their contents. More specifically, there is no array indexing operator.

Pointers

A pointer is a value that refers to another object (or function). You might say it contains the object's address. Here are the operations that pointers support:

1. sizeof – getitssize
   Like arrays, pointers have a size that can be obtained with sizeof. Note that different pointer types can have different sizes.
2. & - get its address
   Assuming your pointer is an lvalue, you can take its address with &. The result is a pointer to a pointer.
3. * - dereference it
   Assuming the base type of your pointer isn't an incomplete type, you can dereference it; i.e., you can follow the pointer and get the object it refers to. Incomplete types includevoid and predeclared struct types that haven't been defined yet.
4. +, - - pointer arithmetic
   If you have a pointer to an array element, you can add an integer amount to it. This amount can be negative, and ptr - n is equivalent to ptr + -n (and -n + ptr, since + is commutative, even with pointers). If ptr is a pointer to the i'th element of an array, then ptr + n is a pointer to the (i + n)'th array element, unless i + n is negative or greater than the number of array elements, in which case the results are undefined. If i + n is equal to the number of elements, the result is a pointer that must not be dereferenced.

Structure	Union
1.The keyword struct is used to define a structure	1. The keyword union is used to define a union.
2. When a variable is associated with a structure, the compiler allocates thememory for each member. The size of structure is greater than or equal to the sum of sizes of its members. The smaller members may end with unused slack bytes.	2. When a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of union is equal to the size of largest member.
3. Each member within a structure is assigned unique storage area of location.	3. Memory allocated is shared by individual members of union.
4. The address of each member will be in ascending order This indicates thatmemory for each member will start at different offset values.	4. The address is same for all the members of a union. This indicates that every member begins at the same offset value.
5 Altering the value of a member will not affect other members of the structure.	5. Altering the value of any of the member will alter other member values.
6. Individual member can be accessed at a time	6. Only one member can be accessed at a time.
7. Several members of a structure can initialize at once.	7. Only the first member of a union can be initialized.

Q.14 Write a program to passing array to function.

Answer:

C programming, a single array element or an entire array can be passed to a function. Also, both one-dimensional and multi-dimensional array can be passed to function as argument.

Passing One-dimensional Array In Function

C program to pass a single element of an array to function

```
#include <stdio.h>
void display(int a)
  {
  printf("%d",a);
  }
int main(){
```

```
  int c[]={2,3,4};
  display(c[2]);  //Passing array element c[2] only.
  return 0;
}
```

Output

4

Single element of an array can be passed in similar manner as passing variable to a function.

Passing entire one-dimensional array to a function

While passing arrays to the argument, the name of the array is passed as an argument(,i.e, starting address of memory area is passed as argument).

Write a C program to pass an array containing age of person to a function. This function should find average age and display the average age in main function.

```
#include <stdio.h>
float average(float a[]);
int main(){
    float avg, c[]={23.4, 55, 22.6, 3, 40.5, 18};
    avg=average(c);   /* Only name of array is passed as argument. */
    printf("Average age=%.2f",avg);
    return 0;
  }
float average(float a[]){
    int i;
    float avg, sum=0.0;
    for(i=0;i<6;++i){
      sum+=a[i];
    }
    avg =(sum/6);
    return avg;
}
```

Output

Average age=27.08

Passing Multi-dimensional Arrays to Function

To pass two-dimensional array to a function as an argument, starting address of memory area reserved is passed as in one dimensional array

Example to pass two-dimensional arrays to function

```
#include
void Function(int c[2][2]);
int main(){
  int c[2][2],i,j;
  printf("Enter 4 numbers:\n");
  for(i=0;i<2;++i)
     for(j=0;j<2;++j){
```

```
        scanf("%d",&c[i][j]);
    }
  Function(c);   /* passing multi-dimensional array to function */
  return 0;
}
void Function(int c[2][2]){
/* Instead to above line, void Function(int c[][2]){ is also valid */
  int i,j;
  printf("Displaying:\n");
  for(i=0;i<2;++i)
    for(j=0;j<2;++j)
       printf("%d\n",c[i][j]);
}
```

Output

```
Enter 4 numbers:
2
3
4
5
Displaying:
2
3
4
5
```

Q.15 Explain call by value and call by reference with an example.

Answer:

These methods are different ways of passing (or calling) data to functions.

Call by Value

If data is passed by value, the data is copied from the variable used in for example main() to a variable used by the function. So if the data passed (that is stored in the function variable) is modified inside the function, the value is only changed in the variable used inside the function. Let's take a look at a call by value example:

```
#include <stdio.h>
void call_by_value(int x) {
        printf("Inside call_by_value x = %d before adding 10.\n", x);
        x += 10;
        printf("Inside call_by_value x = %d after adding 10.\n", x);
}

int main() {
```

```
	int a=10;

	printf("a = %d before function call_by_value.\n", a);
	call_by_value(a);
	printf("a = %d after function call_by_value.\n", a);
	return 0;
}
```

The output of this call by value code example will look like this:
a = 10 before function call_by_value.
Inside call_by_value x = 10 before adding 10.
Inside call_by_value x = 20 after adding 10.
a = 10 after function call_by_value.

Ok, let's take a look at what is happening in this call-by-value source code example. In the main() we create a integer that has the value of 10. We print some information at every stage, beginning by printing our variable a. Then function call_by_value is called and we input the variable a. This variable (a) is then copied to the function variable x. In the function we add 10 to x (and also call some print statements). Then when the next statement is called in main() the value of variable a is printed. We can see that the value of variable a isn't changed by the call of the function call_by_value().

Call by Reference

If data is passed by reference, a pointer to the data is copied instead of the actual variable as is done in a call by value. Because a pointer is copied, if the value at that pointers address is changed in the function, the value is also changed in main(). Let's take a look at a code example:

```
#include <stdio.h>
void call_by_reference(int *y) {
	printf("Inside call_by_reference y = %d before adding 10.\n", *y);
	(*y) += 10;
	printf("Inside call_by_reference y = %d after adding 10.\n", *y);
}
int main() {
	int b=10;

	printf("b = %d before function call_by_reference.\n", b);
	call_by_reference(&b);
	printf("b = %d after function call_by_reference.\n", b);

	return 0;
}
```

The output of this call by reference source code example will look like this:
b = 10 before function call_by_reference.

Inside call_by_reference y = 10 before adding 10.
Inside call_by_reference y = 20 after adding 10.
b = 20 after function call_by_reference.
Let's explain what is happening in this source code example. We start with an integer b that has the value 10. The function call_by_reference() is called and the address of the variable b is passed to this function. Inside the function there is some before and after print statement done and there is 10 added to the value at the memory pointed by y. Therefore at the end of the function the value is 20. Then in main() we again print the variable b and as you can see the value is changed (as expected) to 20.